ロールの事前探索によるREST APIファジング手法の提案

山下 一樹[1,a)]　掛井 将平[1]　齋藤 彰一[1]

概要：多くの Web サービスでは REST API が利用されているが，入力の組合せは膨大であり，人手による網羅的な検査は困難である．そこで，テスト入力を自動生成して異常と脆弱性を探索するファジングが活用されている．一方，同一のリクエストでも，管理者と一般利用者のようなロールによって，正常に処理される場合と認証と認可のエラーとなる場合がある．既存手法ではロールを探索の対象とせず固定するため，他のロールなら到達できるエンドポイントを取りこぼす．本稿では，ファジング前にロールのみを変化させ，各エンドポイントの成功ロールを学習することで，検証可能な API カバレッジを向上する手法を提案する．ロール構造の異なる 3 種の REST API で評価した結果，正常応答に到達したエンドポイント数は 3 種すべてで増加し，うち 1 種では 3 反復の平均で 115 件中 52.3 件から 72.0 件へ増加した．また，全ロールを個別に実行した場合の到達エンドポイントの和集合の約 9 割に，4 分の 1 から 9 分の 1 の時間で到達した．以上から，成功ロールの事前学習により，限られた時間でファジングの API カバレッジを高められることを示した．

キーワード：REST API，ファジング，Web，ロール，カバレッジ

Role-Aware REST API Fuzzing through Preliminary Role Discovery

Kazuki Yamashita[1,a)]　Shohei Kakei[1]　Shoichi Saito[1]

Abstract: Many web services are built on REST APIs, but the number of input combinations is enormous, and exhaustive manual inspection is infeasible. Fuzzing, which automatically generates test inputs and searches for anomalies and vulnerabilities, is therefore used. Meanwhile, even for an identical request, the role—such as an administrator or a general user—determines whether the request is processed successfully or results in an authentication or authorization error. Because existing methods do not treat the role as a target of exploration and keep it fixed, they miss endpoints that other roles can reach. This paper proposes a method that varies only the role before fuzzing and learns, for each endpoint, which role succeeds, thereby improving verifiable API coverage. Evaluating the proposed method on three REST APIs with differing role structures, the number of endpoints reached with a successful response increased on all three; on one of them it rose from 52.3 to 72.0 out of 115. It also reached roughly 90% of the union of the endpoints reached by running every role separately, in one quarter to one ninth of the time. These results show that learning successful roles in advance improves the API coverage of fuzzing within a limited time budget.

Keywords: REST API, fuzzing, web, role, coverage

1. はじめに

多くの Web サービスは，機能を REST API として公開している．Web ページとモバイルアプリを操作すると，多くの場合その背後で 1 つ以上の REST API が呼び出される．バックエンドの開発ではマイクロサービス構成での利用がとくに一般的であり，企業の IT 基盤も REST API に依存している [1]．同時に，REST API は攻撃対象としての比重を高めている．Akamai の観測では，Web アプリケーションと API のエンドポイントを狙う攻撃は 2023 年から 2025 年にかけて 73%増加し，API に対する 1 日あたりの平

[1] 名古屋工業大学
Nagoya Institute of Technology
[a)] cmk13005@nitech.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

均攻撃数は前年比で113%増加している [2]．一方で，REST API が受け取る入力の組合せは膨大であり，人手による網羅的な検査は現実的でない．そこで，テスト入力を自動生成して送信し，応答から異常と脆弱性を探索するファジングが活用されている．ファジングを行うツールをファザーと呼ぶ．REST API を対象とする主なファザーとして，RESTler[3]，RESTTestGen[4]，Morest[5]，EvoMaster[6]，DeepREST[7]，WuppieFuzz[8] が提案されている．

ファジングが API の深い処理へ到達するには，まず正常応答である 2xx を返すリクエスト列を組み立てる必要がある．呼び出し順序が誤っていれば，ファザーが変異させる入力値がどれほど妥当でも対象の処理には届かない．既存研究はこの組み立てを，エンドポイントの一覧と入力の型と応答の形式を機械可読な形式で記述した OpenAPI 仕様から，機械的に読み取れる情報を手がかりに行う．具体的には，リクエスト間のデータ依存を OpenAPI 仕様から推論して呼び出し順序を決める手法 [3], [4], [5] と，OpenAPI 仕様に現れない暗黙の順序制約を実行時の応答から学習する手法 [7] がある．

一方，実運用の API では，同一のリクエストであっても，実行主体によって，正常応答と，認証と認可に関するエラーである 401 または 403 とに分かれる．ファジングにおける認証は，実務と研究の双方で課題として認識されている．Arcuri らは，自動車製造企業へファザーを導入した 1 年間の経験から，実運用の API の多くが認証を要し，既存の利用者の認証情報を与えなければ効果的にファジングできないと述べている [1]．本稿では，この実行主体の違いを認証コンテキストと呼び，個々の実行主体をロールと呼ぶ．未認証の状態も 1 つのロールとして数える．

しかし，既存手法が読み取る OpenAPI 仕様は，アクセスポリシーを表現できない [9]．そのため，どのロールがどのエンドポイントへ到達するかは OpenAPI 仕様から判別できない．仕様から判別できない以上，既存研究においてロールは学習の対象ではなく前提として置かれてきた．その位置づけは，評価の前提から外す研究 [4] と，事前に固定で与える設定として扱う研究 [10] と，ファジング完了後の検査に用いる研究 [9] の 3 通りであり，いずれもロールを，正常系へ到達するために学習すべき条件としては扱っていない．

本稿は，認証コンテキストを，入力値と呼び出し順序と並ぶ正常系到達条件の一つとして位置づけ，これを能動的に学習する手法を提案する．提案手法は 2 つのフェーズからなる．フェーズ 1 では，ファジングに入る前に，シードとなるリクエスト列の入力値と呼び出し順序を変えないまま，ロールだけを変えて実行する．この実行を統制フェーズと呼ぶ．401 または 403 から 2xx への変化を到達性のフィードバックとして扱い，エンドポイントごとにどのロールが 2xx へ到達するかを表す endpoint-role mapping を学習する．フェーズ 2 では，学習した成功ロールを高い確率で選びながら，到達したエンドポイントをフィードバックとして次に送るリクエスト列を選ぶ coverage-guided fuzzing を行う．

本稿の貢献は次の 3 点である．

- 認証コンテキストを，正常系到達条件のうち能動的に学習すべき要素として定型化した．
- 統制フェーズと coverage-guided fuzzing からなる 2 フェーズ手法を提案し，WuppieFuzz[8] 上に実装した．
- ロール構造の異なる 3 種の公開ベンチマークで，単一ロール固定に対する到達エンドポイント数の増加を実測した．あわせて，全ロールを個別に実行した場合との時間効率と，事前知識なしにロールを 1 つ選んだ場合との差を示し，増加量がロール構造に依存する事実を示した．

以降，2 章で前提と用語を整理し，3 章で関連研究における位置づけを述べる．4 章で提案手法と実装を述べ，5 章で 3 種のベンチマークによる評価を示す．6 章で結果を考察して今後の課題を述べ，7 章でまとめる．

2. 準備と問題設定

本章では，REST API ファジングの基本的な流れと本稿における指標を述べ，正常系への未到達要因のうち扱う範囲を示す．

2.1 REST API ファジング

REST API ファジングは，テスト対象の API へリクエストを送り，その応答を観測する過程を反復して異常を探索する．1 つの入力を変異させる通常のファジングと異なり，REST API ではあるリクエストが作成したデータを別のリクエストが参照するため，単独のリクエストではなくリクエストの列を組み立てる必要がある．データを作成するリクエストが先行しなければ，そのデータを参照する後続のリクエストは正常に処理されないためである．

リクエストの宛先であるエンドポイントは，HTTP メソッドと，可変部分を含みうる path テンプレートの組であり，OpenAPI 仕様の 1 操作に対応する．同じ path でもメソッドが異なれば別のエンドポイントとして数え，`GET /consumers` と `POST /consumers` は異なるエンドポイントである．本稿におけるカバレッジは，テスト対象の内部の実行経路を測るコードカバレッジではなく，仕様が列挙するエンドポイントのうちファジングが到達したものを数える API カバレッジである．少なくとも一度 2xx 応答を得たエンドポイントの数を到達数と呼び，正常系にどれだけ到達できたかを表す量として用いる．coverage-guided fuzzing では，観測したカバレッジをフィードバックとして，次に変異させるリクエスト列を選ぶ．

© 2026 Information Processing Society of Japan

2.2 正常系到達を阻む要因と本稿の対象

あるエンドポイントが 2xx に到達しない要因を，本稿では入力値，依存関係，状態，認証コンテキストの 4 つに整理する．入力値とは，リクエストが与えるパラメータとボディの値である．入力値による未到達は，これらの値が OpenAPI 仕様の定める型と書式と値域の制約を満たさない場合に生じる．依存関係とは，先行するリクエストが生成した識別子を後続のリクエストが必要とする関係である．依存関係による未到達は，その先行するリクエストが実行されず，必要な識別子が得られない場合に生じる．状態とは，テスト対象が保持するデータと処理の進行状況である．状態による未到達は，対象の処理が特定の状態遷移を前提とし，その前提が満たされていない場合に生じる．認証コンテキストとは，リクエストの実行主体の違いである．認証コンテキストによる未到達は，入力値と依存関係と状態がすべて成立していても，実行主体であるロールが不適切である場合に生じる．

入力値，依存関係，状態の 3 つは既存研究が繰り返し挙げてきた要因である．Kim らはコードカバレッジが低い要因としてパラメータ値の生成と操作間の依存関係の検出，および仕様と実装の乖離を挙げ [11]，Zhang らはデータベースの状態と，同じデータを扱う複数のエンドポイントの結びつきを未解決の課題として挙げている [12]．これに対し認証は，認証を要するエンドポイントがテスト生成の対象から外れるためにコードカバレッジが低くなると指摘されるものの [13]，他の 3 つと並ぶ到達条件としては整理されていない．本稿は認証コンテキストを 4 つ目の要因として加える．

本稿が直接扱う要因は認証コンテキストである．他の 3 要因の解決は基盤とするファザーに委ねる．

3. 関連研究

本章では，正常系リクエスト列を構築する既存手法を整理し，それらがロールをどう扱ってきたかを述べ，提案手法と先行研究との差分を示す．

3.1 正常系リクエスト列の構築

正常系リクエスト列の構築方法は主に 2 種類ある．一つ目は，仕様の静的解析から系列を組み立てる方法である．RESTler は，一方が返した識別子を他方が入力として要求する producer-consumer 依存を仕様から推論し，実行応答からの動的フィードバックで無効な組合せを除外しながら，ステートフルなリクエスト列を生成する [3]．RESTTestGen は，操作を頂点とし，一方の応答と他方の要求に共通する項目をデータ依存の辺とする operation dependency graph を構築し，実行中に更新しながら正常系と異常系のテストを生成する [4]．Morest は，操作とスキーマを頂点とするグラフモデルである RESTful-service property graph を動的に更新し，呼び出し系列の生成を誘導する [5]．

二つ目は，実行時の応答から学習する方法である．EvoMaster は，コードカバレッジと HTTP ステータスに基づく適応度で進化的探索を行い，テスト対象の内部情報であるホワイトボックス情報を利用してテスト系列を生成する [6]．DeepREST は，OpenAPI 仕様に現れない暗黙の順序制約と入力制約を深層強化学習で獲得し，探索と活用を交互に行う [7]．本稿が基盤とする WuppieFuzz もこれに含まれる [8]．

2 種類の正常系リクエスト列の構築方法はいずれも，呼び出し順序と入力値の精度を高める点に注力しており，ロールは探索の対象に含めていない．

3.2 既存研究におけるロールの扱い

既存研究におけるロールの位置づけは 3 通りに大別できる．第一は，評価の前提から外す研究である．RESTTestGen は，認証を要する API を評価対象から除外し，匿名の利用者には禁止される操作が検査できていない点を妥当性への脅威として自ら挙げている [4]．第二は，事前に固定で与える設定として扱う研究である．Zhang らのツール比較では，5 個の API について認証情報を手作業で設定する必要があり，ツールごとに設定方法が異なると報告されている [12]．Sahin らは，この設定コストと非互換性そのものを REST API ファジング研究の課題の 1 つとして挙げ，認証情報を宣言的に記述する共通形式と，36 個のオープンソース API からなるデータセット WFD を整備した [10]．第三は，ファジング後の検査に用いる研究であり，次節で述べる．

これらの研究では，ロールは与えられる前提か，違反を判定する基準か，評価から外す対象であり，学習して獲得する到達条件としては扱われていない．加えて，入力値を無作為に変異させるファジングが，別のロールの正しい認証情報を偶然生成することは現実的に期待できない [10]．すなわちロールは，入力値の変異によっては得られない到達条件である．

3.3 認可ポリシー違反の検出との対比

本稿の提案手法に最も近い先行研究は，Arcuri らの認可ポリシー違反検出である [9]．同研究は，Broken Object Level Authorization と Broken Function Level Authorization という認可の不備を動機として挙げ，認証と認可の扱いについて，違反を自動で判定する規則であるオラクルを 3 種定義して EvoMaster の拡張として実装している．統合の形態は後処理であり，ファジング完了後に，生成済みのテスト集合を土台として検証用のテストを新たに作る．

本稿は，401 と 403 をフィードバックとして用いる点で先行研究と共通するものの，目的とフェーズは異なる．先

© 2026 Information Processing Society of Japan

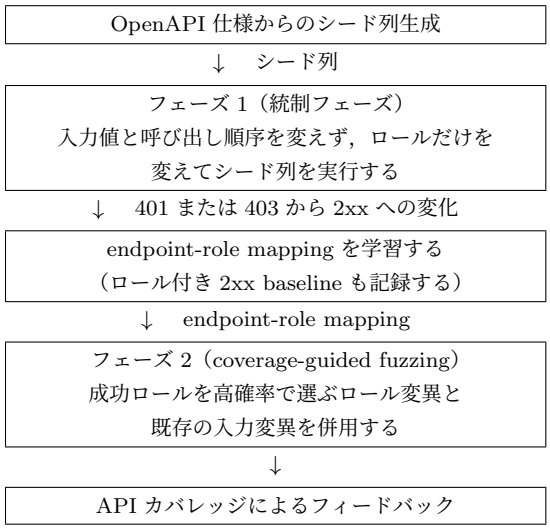


図 1 提案手法の 2 フェーズ構成
Fig. 1 Two-phase structure of the proposed method.

行研究の目的は権限違反の検出であり，ファジング完了後に適用する．本稿の目的は正常系への到達性の学習であり，ファジング開始前の統制フェーズに適用する．先行研究は探索の結果を検査する一方で，本稿は探索の入力となる到達条件そのものを作る．

3.4 本稿の位置づけ

以上より，既存研究はロールを，与えられる前提か，違反を判定する基準か，評価から外す対象として扱ってきた．これに対し本稿は，ロールを API カバレッジを高めるための探索次元として扱う．どのロールがどのエンドポイントへ到達するかをファジング開始前に実行によって学習し，その結果を後段の探索へ引き渡す．この着眼点が既存研究との差分である．

4. 提案手法

本章では，提案手法の全体構成を述べ，各フェーズの手順とデータ構造を示し，ロールの学習を統制フェーズから独立させる理由を述べる．最後に実装を述べる．

4.1 全体構成

提案手法は，**図 1** に示す 2 つのフェーズからなる．フェーズ 1 は統制フェーズであり，シード列の入力値と呼び出し順序を変えずにロールだけを変えた実行により endpoint-role mapping を学習し，2xx 応答を得たリクエスト列にそのときのロールを付与した 2xx baseline を記録する．フェーズ 2 は coverage-guided fuzzing であり，学習した mapping に従ってロールを選びながら探索する．

4.2 フェーズ 1：統制フェーズ

フェーズ 1 では，OpenAPI 仕様から生成したシード列を，設定されたすべてのロールで順に実行する．各実行に際してロール以外は変更しない．依存関係の解決は，そのロール自身の応答から系列ごとに行う．先行するリクエストが遮断されて後続が必要とする値を得られない場合は，その系列の実行をそこで打ち切る．変更がロールだけである以上，観測された応答の違いはロールの違いに帰属できる．

各リクエストの応答は，エンドポイント単位で次のように分類する．2xx を観測したロールは，そのエンドポイントの成功ロールとする．401 または 403 を観測したロールは，認証によって遮断されたロールとする．5xx は，実装の異常を示す可能性はあるものの正常系への到達ではないため，成功として扱わない．それ以外のステータスは記録するだけで，成功とも遮断とも判定しない．

フェーズ 1 は 2 種類の情報を出力する．1 つは endpoint-role mapping であり，エンドポイントごとに成功ロールの集合と遮断ロールの集合を出力する．成功ロールと遮断ロールがともに空でないエンドポイントは，ロールを変えるだけで 401 または 403 から 2xx へ反転する変換候補である．変換候補の件数は，ロール固定によって失われていた到達の量を表す．もう一つの出力は，4.1 節で述べたロール付きの 2xx baseline である．どのロールでどのエンドポイントへ到達できたかを，リクエスト列の単位で出力する．

4.3 フェーズ 2：ロールを含む coverage-guided fuzzing

フェーズ 2 では，リクエスト列に対する既存の変異に加えて，リクエストのロールを書き換える変異を適用する．ロール変異は，対象リクエストのエンドポイントについて mapping が成功ロールを保持していれば，95%の確率でその中から選ぶ．残りの 5%の確率では，設定された全ロールから一様に選ぶ．前者は学習結果の活用であり，後者はフェーズ 1 が取りこぼしたロールの探索である．

ロールを学習結果へ決定的に固定せず確率的に選ぶ理由は，mapping が完全ではないためである．フェーズ 1 が実行するのは仕様由来のシード列に限られ，変異によってのみ到達する状態で成功するロールを観測できない．ロール変異にランダム性を残すことで，入力の変異と同様に仕様外のロール変異を探索する．

4.4 統制フェーズを独立させる理由

ロールの学習は次の 2 つの理由で独立させる．第一は，フィードバックの帰属不能である．探索の途中で 403 を観測しても，その原因が入力値の破壊か依存関係の欠如か状態が満たされていないかロールの不一致かを判別できない．したがって，探索中に観測した 401 と 403 をそのまま到達性のフィードバックとして用いると，誤った学習を招く．入力値と呼び出し順序を変えない統制フェーズの中でのみ，応答の変化をロールに帰属できる．

第二は，探索時間の短縮のためである．成功ロールを

© 2026 Information Processing Society of Japan

表 1 ProxyPrint の `GET /consumer/info` に対する endpoint-role mapping の出力例

Table 1 Example endpoint-role mapping output for `GET /consumer/info` of ProxyPrint.

ロール	観測ステータス	判定
consumer	200	成功
manager	403	遮断
admin	403	遮断
employee	403	遮断
未認証	401	遮断

ファジングの最中に見つける方式では，どのロールが 2xx へ到達するかを引き当てるまでの試行が探索時間を消費する．学習をフェーズ 1 へ前倒しすることで，フェーズ 2 は最初から学習済みの成功ロールを用いて探索できる．探索の途中で成功ロールを見つける場合と比べ，ロールの不確実性に費やす時間を探索へ回せると考えられる．

4.5 実装

提案手法を WuppieFuzz へ統合して実装した．WuppieFuzz は，オープンソースの REST API ファザーである [8]．OpenAPI 仕様からリクエスト列の初期コーパスを生成し，REST 固有の変異と汎用の変異を適用して，測定したカバレッジに基づき次に送るリクエスト列を選ぶ．また，設定ファイルで認証方式を指定できる．本稿は，テスト対象へ計装せず応答だけをフィードバックとするブラックボックスモードを用いる．認証情報は，既定のロールとロールの一覧からなる YAML で与える．各ロールには HTTP Basic，Bearer トークン，Authorization ヘッダへの直接指定，未認証のいずれかの方式を指定できる．

フェーズ 1 は，シード列とロールの直積に対する繰り返し実行として実装した．学習した mapping は CSV と JSON で，ロール付き 2xx baseline はリクエスト列のファイル集合として出力する．CSV は 1 行がエンドポイントとロールの組に対応し，観測したステータスと成功か遮断かの判定を持つ．例えば，評価対象の 1 つである ProxyPrint[14] の `GET /consumer/info` では，consumer だけが成功と判定され，他の 4 ロールは遮断と判定される（**表 1**）．このエンドポイントは 4.2 節で述べた変換候補である．

フェーズ 2 のロール変異は，リクエスト列へ変異を加える部品であるミューテータとして実装した．ロール変異が学習済みの成功ロールから選ぶ確率は 95%であり，残りの 5%を全ロールからの一様な選択に充てる．フェーズ 2 の初期コーパスには，フェーズ 1 と同じく WuppieFuzz 標準の仕様由来コーパスを用いる．

5. 評価

本章では，評価課題を定め，対象と条件と指標を述べ，3 種のベンチマークで得た結果を示す．

表 2 評価対象の REST API

Table 2 REST APIs used in the evaluation.

対象	認証方式	エンドポイント	ロール
ProxyPrint	HTTP Basic	115	5
Scout API	API キー	49	4
Familie BA Sak	OAuth 2.0	183	9

5.1 評価課題

次の 3 つの評価課題を設定する．

RQ1 単一のロールに固定した場合に失われる正常系到達は実在するか．

RQ2 提案手法は，単一のロールに固定した場合に対して到達数を増やすか．

RQ3 提案手法は，ロールを個別に実行する方法に対して，実行時間あたりの到達数を高める効率的な探索を行うか．

5.2 対象と実験条件

評価対象には**表 2** に示す 3 種の REST API を用いる．いずれも公開データセット WFD[10] に含まれる，互いに独立したオープンソースの API である [14], [15], [16]．すべて認証を要するが，その認証方式は異なる．実行に必要な認証情報は，各ベンチマークが試験用に同梱する認証情報と，モック認証サーバが発行するトークンから与える．なお，3 種はいずれもローカルのコンテナ環境に構築し，実行ごとに再起動して初期状態に戻す．コードカバレッジのための計装は行わず，応答のみを観測する．実装の基盤となった WuppieFuzz は乱数種を固定する機能を持たないため，実行ごとに独立した試行である．ファジングを行う実験では，各実行の実行時間を 1 時間とし，条件ごとに 3 反復する．カバレッジを評価する際の基準となる全体数は各仕様の操作数であり，表 2 のエンドポイント列に等しい．一意に定まる path を含む定義と`/api/task/{id}`のように変数を含む定義が重なる場合は，前者を優先して対応づける．

5.3 評価指標

主指標は，2.1 節で定めた到達数である．副指標として，401 または 403 と 2xx の双方が観測されたエンドポイント数を変換数と定義し，ロール固定では失われていた到達の回収量として用いる．ただしこの指標は順序を区別しないため，同一の実行のなかで 2xx から 401 または 403 へ劣化した場合も数える．いずれの指標も，各実行のフェーズ 2 のトラフィックを対象に集計し，フェーズ 1 で得た到達は含めない．

5.4 RQ1：ロール構造の実測

ロールどうしの到達範囲の関係を本稿ではロール構造と呼ぶ．あるロールの到達範囲が他のすべてのロールの到達

© 2026 Information Processing Society of Japan

表 3 ロール構造の実測
Table 3 Measured role structure.

対象	和集合	最良単一	支配差	clean 排他
ProxyPrint /115	78.7	61.0	17.7	9
Scout API /49	24.0	23.0	1.0	0
Familie BA Sak /183	65.0	57.3	7.7	3

範囲を包含する場合を入れ子構造と呼び，そのようなロールが存在しない場合を相補的な構造と呼ぶ．相補的な構造では，どのロールを選んでも他のロールだけが到達するエンドポイントには到達しない．ロール構造は仕様から判別できないため，対象を選定した時点では 3 種の構造は不明である．各ロールを単独で 1 時間ずつ 3 反復実行し，ロール構造を実測した結果を**表 3** に示す．和集合の列は，反復ごとに全ロールの到達エンドポイントの和集合をとり，その件数を 3 反復で平均した値である．支配差は，和集合の件数から最良の単一ロールの到達数を引いた値である．最良の単一ロールは，ProxyPrint が manager，Scout API が administrator，Familie BA Sak が System である．clean 排他は，ちょうど 1 つのロールだけが 2xx に到達し，他のすべてのロールが 401 または 403 のみを観測したエンドポイント数である．clean 排他だけは平均ではなく，3 反復を通じた各ロールの到達エンドポイントの和集合に対する集合演算で求める．2 つの指標は RQ1 の別の側面を示す．支配差が正であれば，最良の単一ロールの到達数が全ロールの和集合に届かないことを意味し，失われる到達の量を表す．clean 排他は，その差に含まれるエンドポイントの一部が入力値と依存関係ではなく認証と認可による遮断に帰せることを直接示す．ロール別の到達数は付録 A.1 に示す．

ProxyPrint では支配差が 17.7 であり，clean 排他を consumer が 5 件，admin が 2 件，manager が 2 件と複数のロールが持っていた．すなわち最上位のロールが存在しない相補的な構造である．

Scout API では，clean 排他がすべてのロールで 0 件であり，支配差も 1.0 にとどまった．administrator だけが到達したエンドポイントは 1 件あったものの，clean 排他の条件を満たさなかった．clean 排他が 0 件であることから，本稿では Scout API を入れ子構造として扱う．

Familie BA Sak はデータベースが空であるため，本来 2xx を返すリクエストの一部が 5xx となり，本稿が示す Familie BA Sak の到達数と変換数はいずれも下界である．支配差は 7.7 であり，clean 排他 3 件を TaskRunner が単独で持っていた．最上位のロールは存在せず，Familie BA Sak も相補的な構造である．この構造は，ファジングとは別に，各ロールが全エンドポイントを 1 回ずつ呼ぶ決定的な走査を Familie BA Sak に対して行った結果とも整合する．この走査では，TaskRunner だけが到達する `/api/task/` 配下の 6 件を 3 反復とも同一に観測した．走査の 6 件が clean 排他 3 件を上回るのは，clean 排他が他のすべてのロールから実際にリクエストが送られたことを要求し，ファジングでは全ロールのうちに 1 時間のあいだ一度も到達を試みないロールが生じうるためである．

以上より，RQ1 の答えは肯定である．3 種すべてで支配差が正であり，最良の単一ロールの到達数でも全ロールの和集合には届かない．相補的な構造の 2 種では，その差に clean 排他が含まれ，未到達の要因が認証と認可による遮断であることまで示せる．入れ子構造の Scout API では支配差が 1.0 で clean 排他は 0 であり，失われる量は小さい．

5.5 RQ2：到達数

単一ロール固定の baseline と提案手法の到達数を比較した結果を**表 4** に示す．値は 3 反復の平均であり，角括弧は最小と最大を示す．

baseline とは，ロール機構を持たない WuppieFuzz 1.3.0 を単一のロールに固定した実行である．固定するロールは，ProxyPrint では consumer，Scout API では user，Familie BA Sak では Veileder とした．提案手法は，baseline と同じ WuppieFuzz 1.3.0 にロール機構を実装した版に，設定されたすべてのロールを与える実行である．提案手法の到達数は，ProxyPrint で 115 件中 72.0 件，Scout API で 49 件中 21.0 件，Familie BA Sak で 183 件中 60.7 件であった．また，baseline と比べてその増加数は，ProxyPrint で 115 件中 19.7 件，Scout API で 49 件中 3.0 件，Familie BA Sak で 183 件中 24.3 件であった．ProxyPrint では 3 反復のいずれの実行でも，提案手法の値が baseline のすべての実行の値を上回った．変換数は，baseline の全実行で 0 であり，提案手法では ProxyPrint で 28.0 件，Scout API で 10.7 件，Familie BA Sak で 60.7 件であった．以上より，RQ2 の答えは肯定である．

増加量はロール構造による差がある．相補的な構造をもつ ProxyPrint と Familie BA Sak では増加が大きく，入れ子構造の Scout API では 3.0 件にとどまった．この差は，表 3 の支配差の大小と整合する．ただし Scout API の 3.0 件は，支配差の 1.0 件を上回った．これは baseline に用いた user が最良の単一ロールではないためであり，この増加は最良ロールとの差ではなく baseline ロールとの差である．

5.6 RQ3：実行時間あたりの到達数

提案手法の実行時間あたりの到達数を検証するために，3 つの指標と提案手法を比較した．第一は全ロールに 1 時間ずつ与えて到達エンドポイントの和集合をとる総当たりであり，対象あたり 4 時間から 9 時間を要する．第二は 1 時間を最良の単一ロールへ与えた場合の到達数である．第三は未認証を含む全ロールの到達数の単純平均である blind 期待値であり，どのロールが最良かを知らずに 1 つ選んだときの期待値に相当する．提案手法の実行時間はフェーズ

© 2026 Information Processing Society of Japan

表 4　単一ロール固定と提案手法の比較
Table 4　Fixed single role vs. the proposed method.

対象	全体数	baseline（固定ロール）	提案手法	増加量	変換数
ProxyPrint	115	consumer 52.3 [52–53]	72.0 [68–74]	+19.7	28.0 [23–31]
Scout API	49	user 18.0 [18–18]	21.0 [20–22]	+3.0	10.7 [10–11]
Familie BA Sak	183	Veileder 36.3 [35–37]	60.7 [58–64]	+24.3	60.7 [58–64]

表 5　ロール別実測を基準とした比較
Table 5　Comparison against per-role executions.

対象	総当たり	和集合	提案	到達率	対最良	対 blind
ProxyPrint	5 時間	78.7	72.0	91.5%	+11.0	+20.1
Scout API	4 時間	24.0	21.0	87.5%	−2.0	+2.8
Familie BA Sak	9 時間	65.0	60.7	93.3%	+3.3	+38.7

1 とフェーズ 2 の合計とし，ロールの学習のために追加の時間を与えない．なお，フェーズ 1 の実測所要は 3 反復の平均で ProxyPrint が 18.0 秒，Scout API が 3.4 秒，Familie BA Sak が 35.3 秒であった．

結果を**表 5** に示す．到達率は，提案手法の到達数を総当たりで得た和集合の件数（表 3 の和集合列）で割った値である．対最良は最良の単一ロールとの差，対 blind は blind 期待値との差であり，いずれも実行時間が等しい 1 時間どうしの差である．

提案手法の到達数は 1 時間の単一の実行時間で，4 時間から 9 時間を要する総当たりの和集合の 87.5%から 93.3%に達した．最良の単一ロールとの比較では，相補的な構造をもつ ProxyPrint で 115 件中 11.0 件，Familie BA Sak で 183 件中 3.3 件，提案手法が上回った．一方，入れ子構造の Scout API では，提案手法が administrator 単独に固定した場合を 49 件中 2.0 件下回った．blind 期待値との比較では，3 種すべてで提案手法が上回った．

以上より，提案手法は総当たりに対して実行時間あたりの到達数を高め，単一のロールへ与える方法に対しても，最良のロールが事前に不明という条件のもとで到達数を高めた．総当たりに対しては，到達数の 9 割前後を 4 分の 1 から 9 分の 1 の実行時間で得た．

6. 考察と今後の課題

本章では，5 章の結果の考察を述べ，今後の課題を示す．

6.1 結果の解釈

5.4 節の clean 排他が示した未到達は，表 4 の変換数としてファジングの規模でも観測され，ロールを探索に加えれば失われていた到達が実際に回収されることを示す．

表 5 の到達率は，提案手法の価値が到達数の上限の更新ではなく，複数ロールの成功条件を 1 つの探索過程へ統合する点にあることを示す．全ロールを個別に実行しても，得られるのはロールごとに分かれた結果であり，ロールをまたいだリクエスト列は生成されない．また単一ロールでの実行では，ロールの違いによって 401 または 403 から 2xx へ反転する様子を原理的に観測できない．実際にロール別実測では，ProxyPrint の 15 回（5 ロール× 3 反復）と Familie BA Sak の 27 回（9 ロール× 3 反復）で変換数が 0 である（付録 A.1）．Scout API のロール別実測で観測された変換数も，401 または 403 から 2xx への反転ではなかった．3 反復を通じた 25 件は，administrator が自らの認証情報に対応するユーザレコードの削除に成功し，その直後から実行終了まで認証を要するリクエストがすべて 401 となった劣化であり，3 反復のうち，自らの認証情報に対応するレコードの削除が成功した 2 回だけ変換数が正である．もう 1 件は，path テンプレートの可変部分が空であったため`/v1/users/{id}`が`/v1/users`として解決されたことによる．一方で本稿が示したのは到達性の改善までであり，到達した正常系からの逸脱を異常として追跡する段階には達していない．

6.2 入れ子構造における劣位

Scout API では，提案手法が最上位ロール固定を 49 件中 2.0 件下回った．支配差が 1.0 件であり clean 排他が 0 件である以上，入れ子構造ではロール次元へ探索の一部を振り向ける利得が小さく，ロールの切替に充てた変異の量だけ単一ロールの深掘りに劣ると解釈できる．ただしこの解釈は未検証であり，ロールへ充てる変異の割合を変えた追試を要する．

もっとも，最上位ロール固定が優位に立つのは，どのロールが最上位かを事前に知っている場合に限られる．1 章と 5.4 節で述べたとおり，ロール構造は仕様から事前に判別できない．相補的な構造では最上位ロール自体が存在しない．したがって現実的な比較対象は事前知識なしにロールを 1 つ選んだ場合であり，この比較では 3 種すべてで提案手法が上回る．

6.3 セキュリティ検査における到達数の意味

到達数の増加は，検査できる処理がそれだけ増えることを意味する．Scout API の`DELETE /v1/users/{id}`に対し，user に固定した baseline は 3 反復とも 2xx を一度も観測せず，削除の処理は一度も実行されない．administrator では 3 反復とも削除の成功を示す 204 に到達し，削除の実装が検査の対象になる．認可の不備を検出するオラクルは，あるユーザで 2xx を得たテストを起点に，同じ呼び出

© 2026 Information Processing Society of Japan

しを別のユーザへ差し替えて応答を比べるため，2xx へ到達したテストがなければ適用できない．Arcuri らは，自身のオラクルについて，ファジングのカバレッジの向上が新たな脆弱性の検出につながりうるとも述べている [9].

6.4 今後の課題

今後の課題は 2 点である．第一に，学習したロール構造に応じて探索の配分を変える機構の実装である．本稿の実装は，ロール変異を他の変異と同じ一定の割合で適用し，探索へ回す確率も定数として与えており，いずれも構造に応じて調整しない．入れ子構造で最上位ロール固定に劣ったのは，切替の利得が小さい構造に対しても同じ割合の変異をロール次元へ充てたためと考えられる．フェーズ 1 の mapping はエンドポイントごとに成功ロールと遮断ロールの集合を保持しており，ロール構造を示す量を導けるため，構造に応じた配分の入力として利用する余地がある．

第二に，フェーズ 1 が確立したロール付き 2xx baseline を異常の判定へ用いる機構の実装である．本稿はロールを到達のための手段として用いたが，ロールごとの正常な応答が既知であれば，そこからの逸脱を異常とみなす基準が得られる．

7. まとめ

本稿は，REST API ファジングにおいて，実行主体の違いである認証コンテキストを，能動的に学習すべき正常系到達条件として位置づけた．ファジング開始前にシード列の入力値と呼び出し順序を変えずロールだけを変える統制フェーズで endpoint-role mapping を学習し，学習した成功ロールで後段の coverage-guided fuzzing を行う 2 フェーズ手法を提案し，WuppieFuzz 上に実装した．

ロール構造の異なる 3 種の公開ベンチマークで 1 時間の実行を 3 反復して評価した結果，単一ロール固定に対する到達数は，ProxyPrint で 115 件中 19.7 件，Scout API で 49 件中 3.0 件，Familie BA Sak で 183 件中 24.3 件増加した．また 1 時間の単一の実行時間で，全ロールを個別に実行して得られる到達エンドポイントの和集合の 87.5%から 93.3%に達した．入れ子構造の Scout API では，最上位ロール固定を 49 件中 2.0 件下回った．ただし，どのロールが最上位かは仕様から事前に判別できず，事前知識なしにロールを 1 つ選ぶ現実的な条件のもとでは，提案手法は 3 種すべてで到達数を上回る結果を示した．

参考文献

[1] Arcuri, A., Poth, A. and Rrjolli, O.: Introducing Black-Box Fuzz Testing for REST APIs in Industry: Challenges and Solutions, *Proc. IEEE Conference on Software Testing, Verification and Validation (ICST)*, pp.382–393 (2025).

[2] Akamai Technologies, Inc.: Apps, APIs, and DDoS 2026: Prepare for the Convergence Crisis: Mitigating API, AI, and DDoS Risks, *State of the Internet*, Vol.12, No.1 (2026).

[3] Atlidakis, V., Godefroid, P. and Polishchuk, M.: RESTler: Stateful REST API Fuzzing, *Proc. IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pp.748–758 (2019).

[4] Viglianisi, E., Dallago, M. and Ceccato, M.: RESTTestGen: Automated Black-Box Testing of RESTful APIs, *Proc. IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, pp.142–152 (2020).

[5] Liu, Y., Li, Y., Deng, G., Liu, Y., Wan, R., Wu, R., Ji, D., Xu, S. and Bao, M.: Morest: Model-based RESTful API Testing with Execution Feedback, *Proc. IEEE/ACM 44th International Conference on Software Engineering (ICSE)*, pp.1406–1417 (2022).

[6] Arcuri, A.: RESTful API Automated Test Case Generation, *Proc. IEEE International Conference on Software Quality, Reliability and Security (QRS)*, pp.9–20 (2017).

[7] Corradini, D., Montolli, Z., Pasqua, M. and Ceccato, M.: DeepREST: Automated Test Case Generation for REST APIs Exploiting Deep Reinforcement Learning, *Proc. 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp.1383–1394 (2024).

[8] Rooijakkers, T., Nijsten, A., Daniele, C., Weitenberg, E., Groenewegen, R. and Melissen, A.: WuppieFuzz: Coverage-Guided, Stateful REST API Fuzzing, *Proc. 12th International Conference on Information Systems Security and Privacy (ICISSP)*, Vol.2, pp.221–231 (2026).

[9] Arcuri, A., Sahin, O. and Zhang, M.: Fuzzing for Detecting Access Policy Violations in REST APIs, *Proc. IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, pp.130–141 (2025).

[10] Sahin, O., Zhang, M. and Arcuri, A.: WFC/WFD: Web Fuzzing Commons, Dataset and Guidelines to Support Experimentation in REST API Fuzzing, arXiv preprint arXiv:2509.01612, dataset DOI 10.5281/zenodo.6106829 (2025).

[11] Kim, M., Xin, Q., Sinha, S. and Orso, A.: Automated Test Generation for REST APIs: No Time to Rest Yet, *Proc. 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pp.289–301 (2022).

[12] Zhang, M. and Arcuri, A.: Open Problems in Fuzzing RESTful APIs: A Comparison of Tools, *ACM Transactions on Software Engineering and Methodology*, Vol.32, No.6, Article 144 (2023).

[13] Ehsan, A., Abuhaliqa, M. A. M. E., Catal, C. and Mishra, D.: RESTful API Testing Methodologies: Rationale, Challenges, and Solution Directions, *Applied Sciences*, Vol.12, No.9, 4369 (2022).

[14] ProxyPrint: proxyprint-kitchen, GitHub (online), available from ⟨https://github.com/ProxyPrint/proxyprint-kitchen⟩ (accessed 2026-08-20).

[15] Svensson, M.: Scout API, included in the Web Fuzzing Dataset (WFD)[10].

[16] NAV: familie-ba-sak, GitHub (online), available from ⟨https://github.com/navikt/familie-ba-sak⟩ (accessed 2026-08-20).

© 2026 Information Processing Society of Japan

付　　録

A.1　ロール別の到達数

5.4 節で用いたロール別の実測値を**表 A·1** に示す．いずれも WuppieFuzz を各ロール単独で 1 時間ずつ 3 反復実行した結果であり，平均と最小と最大を示す．ProxyPrint と Scout API では，ロール別実測は表 4 の baseline とは別の実験であり，同じロール名でも値は一致しない．Familie BA Sak の Veileder の行は，表 4 の baseline と同一の実行である．Familie BA Sak の下位 3 ロールが 1.0 にとどまるのは，データベースが空であるため共有の 1 エンドポイントにしか到達しないことによる．変換数の列は，各ロール単独の実行で 401 または 403 と 2xx の双方が観測されたエンドポイント数であり，ProxyPrint と Familie BA Sak では全ロール全反復で 0 である．Scout API の administrator と user で 0 でないのは 401 または 403 から 2xx への反転によるものではなく，その理由は 6.1 節で述べる．

表 A·1　ロール別の到達数と変換数（1 時間 ×3 反復）
Table A·1　Per-role 2xx endpoint coverage and conversions (three 1-hour repetitions).

対象	ロール	到達数	変換数
ProxyPrint	manager	61.0 [60–62]	0
/115	consumer	54.0 [52–56]	0
	employee	51.7 [51–52]	0
	admin	48.7 [48–49]	0
	未認証	44.0 [44–44]	0
Scout API	administrator	23.0 [22–24]	8.3 [0–13]
/49	moderator	21.3 [21–22]	0
	user	17.7 [17–18]	0.3 [0–1]
	未認証	10.7 [10–11]	0
Familie BA Sak	System	57.3 [56–59]	0
/183	Beslutter	45.3 [45–46]	0
	Saksbehandler	44.3 [43–45]	0
	Veileder	36.3 [35–37]	0
	TaskRunner	7.0 [7–7]	0
	未認証	4.0 [4–4]	0
	Forvalter	1.0 [1–1]	0
	Kode6	1.0 [1–1]	0
	Kode7	1.0 [1–1]	0

© 2026 Information Processing Society of Japan